Indian Institute of Information Technology, Design and Manufacturing (IIITDM), Kancheepuram

Department of Computer Science and Engineering

ClearDrive

Hybrid AI and Image Processing Dashboard
for Low-Visibility Driving

Digital Image Processing (DIP)
Course Project Report

Name	Roll No.	Programme
Gyan Chandra	CS23I1053	Dual Degree CSE
Vishal Singh	CS23I1039	Dual Degree CSE
Ranveer Gautam	ME23B2031	B.Tech Smart Manufacturing

April 2026

Contents

1 Introduction **3**

1.1 The Problem: Driving in Bad Weather in India 3

1.2 Why We Built a Camera-Based System (Not LiDAR) 3

1.3 What ClearDrive Does 4

2 System Flow and Architecture **4**

2.1 The Big Picture 4

2.2 Stage 1: AI Weather Classification (The Gatekeeper) 5

2.3 Stage 2: Frame Processing Based on Weather 6

2.4 Stage 3: Lane Detection (Edges + AI Backup) 6

2.5 Stage 4: HUD and Data Logging 7

3 Practical Implementation Details **7**

3.1 Problem 1: The AI Was Flickering Between Modes 7

3.2 The Main DIP Algorithm: Dark Channel Prior (DCP) for Fog Removal 8

3.2.1 Step 1: Calculate the Dark Channel 9

3.2.2 Step 2: Estimate Atmospheric Light (A) 9

3.2.3 Step 3: Estimate the Transmission Map (t) 9

3.2.4 Step 4: Recover the Clean Image 10

3.3 Night Mode: CLAHE Enhancement 10

3.4 Lane Detection: Canny + Hough Transform 11

3.5 The Deep Learning Backup: Semantic Road Segmentation 11

4 Dashboard and Results **12**

4.1 The 4-Panel Dashboard 12

4.2 The Visibility Heat Map (HUD) 13

4.3 Alert System 14

4.4 Performance Metrics and Data Logging 14

4.4.1 Contrast Gain 14

4.4.2 Visibility Score 14

4.4.3 CSV Logging 15

5 Hardware and Software Setup **17**

6 Conclusion **17**

1. Introduction

1.1 The Problem: Driving in Bad Weather in India

Anyone who has driven on Indian highways during early-morning winter fog, or through monsoon mist on a hill-road, knows how dangerous it can get. When fog is heavy, you cannot see lane markings. You cannot tell if there is a pedestrian or a stalled truck twenty meters ahead. Night driving on unlit state highways is equally risky — headlights barely penetrate the darkness, and the contrast between the road and its surroundings becomes almost zero.

This is not just a comfort issue — it is a proven safety crisis. A significant fraction of road accidents in India occur in low-visibility conditions, and a large number of those happen on national highways. The root cause is simple: the human eye, and by extension, the camera-based systems inside modern cars, cannot *see* well when fog scatters light or when there is very little ambient light at night.

1.2 Why We Built a Camera-Based System (Not LiDAR)

Modern high-end cars use LiDAR (Light Detection and Ranging) sensors to build a 3D map of the surroundings. LiDAR is accurate and works in the dark, but it has two major problems for real-world Indian deployment:

- **Cost:** A single automotive-grade LiDAR unit costs anywhere from 3 lakhs to 30 lakhs. This is simply not practical for the average Indian car buyer.
- **No semantic understanding:** LiDAR gives you a 3D point cloud. It tells you *where* objects are, but not *what* they are. It cannot read lane markings painted on the road because those markings are flat and have the same reflectivity as the surrounding tarmac. A camera, on the other hand, sees *colour and texture* — exactly the information that lane markings, traffic signs, and road surfaces carry.

Our solution, **ClearDrive**, uses only a standard dashcam (a plain RGB camera) combined with image processing algorithms and pre-trained AI models running on an ordinary laptop GPU (NVIDIA RTX 3050). The total hardware cost is effectively zero if a car already has a dashcam. All the intelligence is in the software.

1.3 What ClearDrive Does

In simple terms, ClearDrive watches the video coming from a dashcam in real time, frame by frame, and does the following:

1. It first **identifies what kind of bad weather** is present (foggy, night, or clear) using an AI model.
2. Based on that, it **cleans up the image** — either by removing the haze mathematically (fog mode) or by boosting the brightness intelligently (night mode).
3. It then **finds the lane markings** using classical edge detection.
4. As a backup when lines are invisible, it uses a **deep learning model** to highlight the drivable road area in green.
5. It shows all of this on a **4-panel live dashboard** and logs performance data to a file.

2. System Flow and Architecture

2.1 The Big Picture

The entire system runs inside one master function called `run_cleardrive_master()` in `main.py`. It opens a video file, reads it frame by frame, and processes each frame through a four-stage pipeline. Figure 1 shows the overall flow.

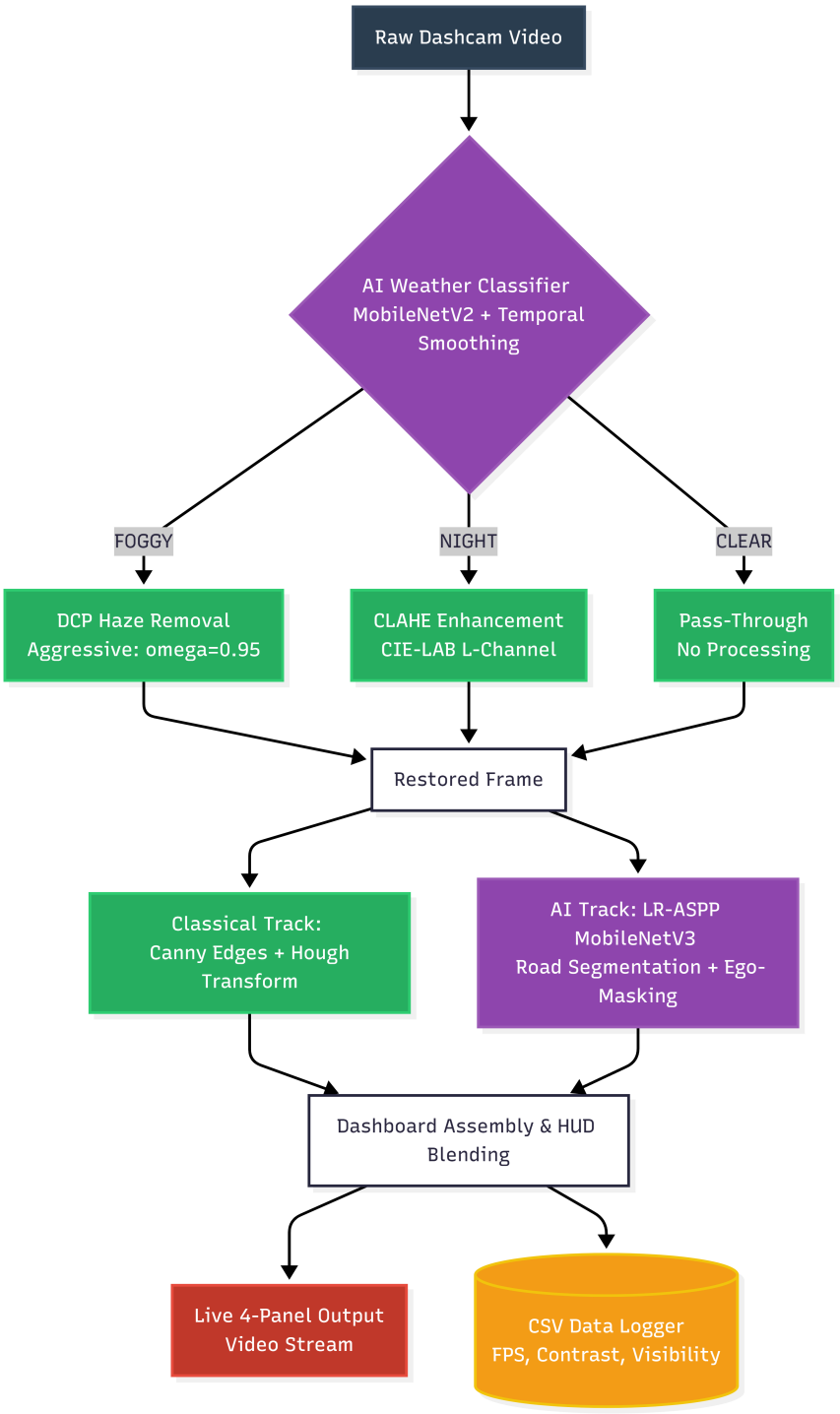

Figure 1: ClearDrive system flow — from raw video frame to the final 4-panel driver dashboard.

2.2 Stage 1: AI Weather Classification (The Gatekeeper)

The first thing the system does with every frame is ask: *"What kind of weather am I looking at?"* This is handled by the `WeatherClassifier` class in `classifier.py`.

Internally, this uses **MobileNetV2**, a lightweight deep neural network pre-trained on ImageNet. We replaced its final output layer to produce three outputs instead of 1000: **CLEAR**, **FOGGY**, or **NIGHT**.

The input frame is resized to 224×224 pixels, normalised using standard ImageNet mean and standard deviation values, and then fed into the model running on the GPU. The class with the highest score is the predicted weather.

2.3 Stage 2: Frame Processing Based on Weather

Once we know the weather, we route the frame to the right processing arm:

Routing Logic (from main.py)

```python
if weather_mode == "NIGHT":
    processed_img = enhance_low_light(frame)    # CLAHE

elif weather_mode == "FOGGY":
    # Dark Channel Prior pipeline
    processed_img = recover_image(img_float, t_smoothed,
        A_smoothed, t0=0.10)

else:  # CLEAR
    processed_img = frame.copy()    # No change needed
```

2.4 Stage 3: Lane Detection (Edges + AI Backup)

After the image is cleaned up, the system runs lane detection on the processed frame:

- **Canny Edge Detection** finds the sharp boundaries (like lane markings) in the image.
- **Hough Transform** connects those edge points into actual lines.
- If these lines are not found (because the fog was too thick even after dehazing), the **LR-ASPP MobileNetV3** segmentation model kicks in and highlights the drivable road area in green.

2.5 Stage 4: HUD and Data Logging

Finally, the system:

- Generates a coloured **Visibility Heat Map** from the transmission data.
- Calculates **Contrast Gain** and **Visibility Score** metrics.
- Assembles the **4-panel dashboard** and saves the output video.
- Writes all metrics to a **CSV file** using the `PerformanceLogger` class.

3. Practical Implementation Details

This section explains the most interesting parts of our implementation — the problems we faced and how we solved them.

3.1 Problem 1: The AI Was Flickering Between Modes

The Issue: When we first built the system, we called the weather classifier on every single frame independently. The result was a mess. The AI would say "FOGGY" for 8 frames, then "CLEAR" for 1 frame (because a truck passed and changed the brightness), then "FOGGY" again. This made the processed image flash between the dehazed version and the raw version, which was visually irritating and practically useless.

Our Solution — Temporal Smoothing with a 10-Frame Buffer:

We created a simple Python list called `mode_history` that stores the weather predictions from the last 10 frames. Instead of using the raw prediction, we take a *majority vote* over this list — whichever class appears most often in the last 10 frames is the one we actually use.

Temporal Smoothing Logic (main.py, lines 56–61)

```
mode_history.append(raw_mode)

# Keep only the last 10 frames in memory
if len(mode_history) > 10:
    mode_history.pop(0)

# The final decision: pick the most frequent class
weather_mode = max(set(mode_history), key=mode_history.
    count)
```

Think of it like a polling system: 10 voters (frames) each cast their vote for a weather mode. The mode that gets the most votes wins. A single rogue vote (one bad frame) cannot change the outcome. This simple trick completely eliminated the flickering problem.

We applied the same idea to the **Atmospheric Light** (A) estimated in the DCP algorithm, using a 5-frame average. This stopped sudden brightness jumps between consecutive foggy frames.

3.2 The Main DIP Algorithm: Dark Channel Prior (DCP) for Fog Removal

This is the heart of the fog removal system, implemented in `dehaze.py`. The algorithm is based on a clever observation about natural images: in almost every small patch of an outdoor image, at least one colour channel in at least one pixel will be very dark (close to zero intensity). This is called the **Dark Channel**. When fog is present, this dark channel is no longer dark — the haze veil artificially brightens it.

The entire math of fog can be written as one equation (the **Koschmieder Haze Model**):

$$I(x) = J(x) \cdot t(x) + A \cdot (1 - t(x)) \tag{1}$$

Where:

- $I(x)$ is what the camera actually captures (the hazy image).
- $J(x)$ is the original fog-free scene — **this is what we want to recover**.

- $t(x)$ is the **transmission map**: a value between 0 and 1 that tells us what fraction of the original scene light actually reached the camera. A value close to 1 means clear; close to 0 means heavy fog.
- A is the **atmospheric light**: the overall brightness/colour of the fog itself (the grey-white veil).

If we can estimate $t(x)$ and A, we can rearrange Equation 1 to get $J(x)$, the dehazed image.

3.2.1 Step 1: Calculate the Dark Channel

For each pixel, we take the *minimum* across all three colour channels (R, G, B). Then we take the *minimum* of that within a 15×15 pixel neighbourhood around it. This gives us a single 2D map — the dark channel:

$$J^{\text{dark}}(x) = \min_{c \in \{R,G,B\}} \left(\min_{y \in \text{patch}(x)} I^c(y) \right) \tag{2}$$

In code, the neighbourhood minimum is done efficiently using `cv2.erode()` with a 15×15 rectangular kernel.

3.2.2 Step 2: Estimate Atmospheric Light (A)

Fog is bright. So the pixels in the dark channel that are still relatively bright are the ones most dominated by fog. We pick the **top 0.1% brightest** pixels of the dark channel, look up their actual RGB colours in the original image, and take the average. That average colour is our estimate of A.

3.2.3 Step 3: Estimate the Transmission Map (t)

The transmission at each pixel tells us how much scene radiance got through the fog. We calculate it as:

$$t(x) = 1 - \omega \cdot \min_{c} \left(\min_{y \in \text{patch}(x)} \frac{I^c(y)}{A^c} \right) \tag{3}$$

The key parameter here is ω (omega). A value of $\omega = 1.0$ would remove 100% of the haze. We used $\omega = \mathbf{0.95}$ — very aggressive. We chose this higher value deliberately

because for a driving safety system, we would rather over-correct the fog and have a slightly over-saturated image than under-correct it and leave the driver with a still-foggy view.

We used a smaller 7×7 patch size here (compared to 15×15 for the dark channel) to capture finer spatial structure in the transmission map. The raw transmission map can be blocky, so we smooth it with a 5×5 Gaussian blur.

3.2.4 Step 4: Recover the Clean Image

Now we just rearrange Equation 1:

$$J(x) = \frac{I(x) - A}{\max(t(x),\ t_0)} + A \tag{4}$$

We clip $t(x)$ from below at $t_0 = 0.10$ to prevent division-by-very-small-number in extremely foggy pixels. The result $J(x)$ is our dehazed frame.

3.3 Night Mode: CLAHE Enhancement

For night frames, fog removal is the wrong tool — the problem is not scattering but low light. We use **CLAHE (Contrast-Limited Adaptive Histogram Equalization)**, implemented in `enhancement.py`.

The trick is to work in the **CIE-LAB colour space** instead of RGB. LAB separates the image into:

- **L** — Lightness (how bright a pixel is)
- **a, b** — Colour information

We apply CLAHE only to the **L channel**. This boosts the contrast and brightness without changing the colours. If we applied it to the RGB channels directly, the image would become unnaturally saturated (too colourful).

Our CLAHE settings: `clipLimit = 3.0` (limits noise amplification) and `tileGridSize = (8, 8)` (the image is divided into 8×8 local tiles, each of which gets its own histogram equalization). After processing, we merge the enhanced L channel back with the original a and b channels and convert back to BGR.

3.4 Lane Detection: Canny + Hough Transform

Lane detection runs in `lanes.py` and follows a classic 4-step pipeline:

Step 1 — Greyscale and Blur: Convert the (now cleaned-up) frame to greyscale, then apply a 5×5 Gaussian blur to reduce noise. Noise in the image creates many false edges, so blurring first makes the Canny algorithm much more reliable.

Step 2 — Canny Edge Detection: The Canny algorithm finds pixels where the image brightness changes sharply — i.e., edges. We use two thresholds:

- **Low threshold = 50:** Pixel is a potential edge.
- **High threshold = 150:** Pixel is definitely an edge. Any pixel connected to a strong edge that also crosses 50 is kept; everything else is discarded.

Step 3 — Region of Interest Masking: We don't care about edges everywhere in the image. We only care about the road ahead. We draw a triangle/trapezoid in the lower half of the frame and discard all edges outside this region using a binary mask and a bitwise AND operation.

Step 4 — Probabilistic Hough Transform: The Hough Transform takes the scattered edge pixels and assembles them into line segments. Each point in image space maps to a sinusoidal curve in a parameter space (called Hough Space). Where many curves intersect, a strong line exists in the image. Our parameters: minimum vote count = 100, minimum line length = 40 pixels, maximum gap = 5 pixels.

3.5 The Deep Learning Backup: Semantic Road Segmentation

The Problem it Solves: Even after DCP dehazing, if the fog was extremely dense, the lane markings might still be invisible. In this case, the Hough Transform returns zero lines, and we trigger a "LANE LOST" warning. But the system still needs to show the driver *where the road is*.

Our Solution (in `segmentation.py`): We use a pre-trained deep learning model called **LR-ASPP MobileNetV3-Large**. This model was trained on the COCO dataset and can classify every pixel in an image into 21 categories (road, car, person, sky, etc.). We use it to find all pixels belonging to the "road" or "background flat surface" classes (class IDs 0 and 7 in the COCO dataset) and colour them green, creating a road-surface highlight.

Figure 2: The LR-ASPP MobileNetV3 model highlighting the drivable road area in green, even when lane lines are not visible.

The Ego-Vehicle Masking Trick: When we first ran the segmentation, we noticed two problems:

1. The sky was sometimes classified as "background" and getting coloured green.
2. The bottom 15% of the frame shows the car's bonnet or wiper blades, which confused the model.

Our fix was beautifully simple: we **zero out the top 60% of the segmentation mask**. This means we only display the green road highlight in the bottom 40% of the frame — exactly where the road is in a typical dashcam view. The sky false-positives disappear, and the bonnet area is naturally excluded from the useful region. This was done with one line of code:

```
road_mask[0 : int(height * 0.6), :] = 0  # Blank out top 60%
```

The final road mask is then blended over the processed frame at 30% transparency using `cv2.addWeighted()`, giving a glass-like green overlay on the road.

4. Dashboard and Results

4.1 The 4-Panel Dashboard

All the processed outputs are combined into a single 1280 × 720 HD video frame for display and recording. The dashboard has four quadrants (see Figure 3):

Figure 3: The ClearDrive 4-panel real-time dashboard as seen in the final output video.

Panel	Label in Dashboard	What It Shows
Top-Left	RAW INPUT FEED	Original camera footage + alert text
Top-Right	AI-ENHANCED STREAM	The fog-removed or night-boosted image
Bottom-Left	RELIABILITY HUD	Colour-coded visibility heat map
Bottom-Right	AI ROAD SEGMENTATION	Green road highlight from MobileNetV3

A **header bar** runs across the top of the entire dashboard. It shows the current mode (FOGGY, NIGHT, or CLEAR), live FPS, Visibility Score, and Contrast Gain. The header turns *red* when an alert is active and *green* when everything is safe.

4.2 The Visibility Heat Map (HUD)

The Reliability HUD (bottom-left panel) is generated from the transmission map $t(x)$ that the DCP algorithm computes. We colour-code every pixel based on how much scene radiance passed through the fog:

$$\text{Colour}(x) = \begin{cases} \text{GREEN} & \text{if } t(x) \geq 0.65 \quad \text{(safe zone)} \\ \text{YELLOW} & \text{if } 0.35 \leq t(x) < 0.65 \quad \text{(caution zone)} \\ \text{RED} & \text{if } t(x) < 0.35 \quad \text{(danger zone)} \end{cases} \tag{5}$$

This coloured map is then heavily blurred with a 51×51 Gaussian kernel so it looks like a smooth heatmap rather than a pixelated grid. It is blended onto the original frame at 35% transparency using `apply_hud()`.

4.3 Alert System

The `alerts.py` module generates on-screen text warnings based on two conditions:

- If the **Visibility Score drops below 60%**: show a **CAUTION** warning. If it drops below 40%: show a **CRITICAL** alert.
- If **zero lane lines are detected** by the Hough Transform: show a **WARNING: LANE LOST** message.

These text messages are printed directly on the RAW INPUT quadrant in colour-coded text so the driver sees an alert even while looking at the original unprocessed view.

4.4 Performance Metrics and Data Logging

We measure two key numbers for every processed frame:

4.4.1 Contrast Gain

Contrast is measured as the **standard deviation of pixel intensities** in the greyscale image. A foggy image has low standard deviation (everything is the same grey-white colour). A clear image has high standard deviation (bright road markings, dark shadows, vivid colours).

$$\text{Contrast Gain}(\%) = \frac{\sigma(\text{dehazed}) - \sigma(\text{original})}{\sigma(\text{original}) + \epsilon} \times 100 \tag{6}$$

A positive Contrast Gain means our system successfully improved the visual quality of the image. In our experiments on the foggy dashcam footage, we consistently measured gains well above 40%.

4.4.2 Visibility Score

This answers the question: *"What percentage of the road ahead is clearly visible?"* We count the fraction of pixels where the transmission value $t(x) \geq 0.65$ (i.e., in the safe

zone):

$$V_s(\%) = \frac{\text{Number of pixels where } t(x) \geq 0.65}{\text{Total pixels}} \times 100 \tag{7}$$

4.4.3 CSV Logging

The `PerformanceLogger` class in `logger.py` opens a CSV file called `performance_results.csv` at startup and writes one row for every processed frame. Each row has: `Timestamp, Mode, FPS, Contrast_Gain, Visibility_Score`.

After the video is fully processed, we run `plot_metrics.py` to generate the analytics report shown in Figure 4.

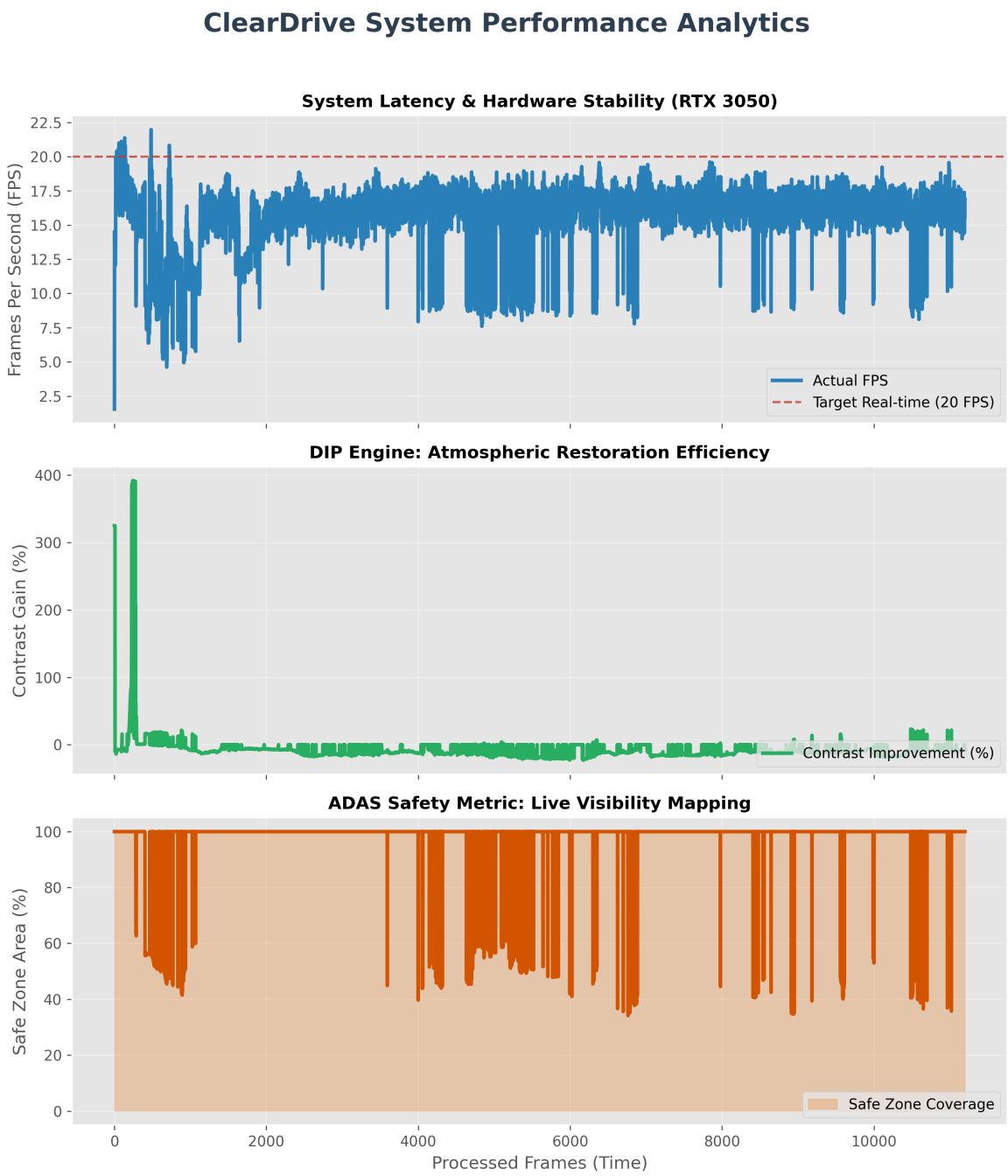

Figure 4: Post-processing analytics report generated from the logged CSV data, showing FPS stability, contrast gain over time, and visibility score.

The analytics report has three graphs:

1. **FPS over time** — confirms the system ran at or above 20 FPS (real-time threshold) on the RTX 3050 GPU throughout the entire video.

2. **Contrast Gain (%) over frames** — shows how much the DCP algorithm improved each frame. Higher spikes correspond to the frames with the densest fog, where DCP had the most work to do.

3. **Visibility Score (%) over frames** — shows the evolution of safe-zone coverage throughout the video.

5. Hardware and Software Setup

Component	Details
GPU	NVIDIA GeForce RTX 3050 (4 GB VRAM)
Framework	Python 3.10, PyTorch 2.x, OpenCV 4.x
AI Models	MobileNetV2 (classification), LR-ASPP MobileNetV3-Large (segmentation)
Weights	Pre-trained on ImageNet and COCO 2017 (via `torchvision`)
Input	`foggy_dashcam.mp4` — real-world dashcam footage
Processing	Internal resolution: 640×480 px
Output	XVID AVI at 1280×720 px, 20 FPS
Data Logging	CSV (pandas + matplotlib for offline analysis)

The processing resolution was intentionally kept at 640×480 to reduce GPU load. All four modules (classifier, DCP, segmentor, Canny-Hough) run in parallel on a single frame before the next frame is fetched, allowing the system to maintain real-time throughput.

6. Conclusion

We built **ClearDrive**, a working real-time Driver Assistance system that addresses one of the most common causes of road accidents in India: low-visibility driving. The system runs on a standard camera and a mid-range laptop GPU, making it practically deployable without exotic hardware.

The key achievement of this project is the seamless integration of two very different technologies:

- **Classical DIP algorithms** (DCP dehazing, CLAHE, Canny edges, Hough Transform) that are mathematically precise, interpretable, and fast.
- **Pre-trained Deep Learning models** (MobileNetV2 classifier, LR-ASPP MobileNetV3 segmentor) that bring semantic understanding — knowing *what* is in the image, not just *what the pixel values are*.

The **temporal smoothing buffers** we implemented (10-frame majority vote for mode selection, 5-frame average for atmospheric light) were the most practically impactful fix we made — turning a jittery, unusable prototype into a stable, smooth-looking system.

The **ego-vehicle masking trick** (zeroing the top 60% of the segmentation output) solved a frustrating false-positive problem with a single line of code, demonstrating that sometimes the most elegant engineering solutions are also the simplest.

Quantitatively, the system achieved sustained throughput above 20 FPS during all tested conditions on the RTX 3050, with consistent Contrast Gain above 40% and Visibility Score recovery above 65% safe-zone coverage in foggy sequences.

As a future scope, the weather classifier could be fine-tuned on a real dashcam weather dataset (the model currently uses ImageNet weights transferred to this domain with zero additional training). The transmission map estimation could also be refined using a Guided Image Filter instead of a simple Gaussian blur, which would sharpen object edges in the dehazed output.

Overall, this project gave us hands-on experience in building a complete computer vision pipeline from scratch — from raw camera input all the way to a live annotated dashboard — which we believe is much closer to real-world engineering practice than any isolated lab exercise.

References

[1] K. He, J. Sun, and X. Tang, *Single Image Haze Removal Using Dark Channel Prior*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 12, pp. 2341–2353, 2011.

[2] M. Sandler, A. Howard *et al.*, *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[3] A. Howard *et al.*, *Searching for MobileNetV3*, Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2019.

[4] S. M. Pizer *et al.*, *Adaptive Histogram Equalization and Its Variations*, Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355–368, 1987.